In [ ]:
```python
import marimo as mo

mo.md(
    """
    # TYK2 ddG: are the extreme values real?

    The dual-state ddG run (`data/ddg/`) produces values down to **-3000**, far outside any
    physical folding free energy. This notebook reproduces the pipeline on one structure and
    tests whether those numbers are **steric clash artefacts** rather than energies.

    Three protocols on the same relaxed pose:

    | | what it does |
    |---|---|
    | **A** | the pipeline: `mutate_residue(pack_radius=8)`, side chains only |
    | **fa_rep** | how much of the ddG is the Lennard-Jones repulsive term |
    | **B** | A plus backbone + side-chain minimisation of the 8 A shell |

    Sign convention follows the paper: **ddG = E(WT) - E(mut)**, so *negative = destabilising*.
    """
)
```

TYK2 ddG: are the extreme values real?

The dual-state ddG run (`data/ddg/`) produces values down to **-3000**, far outside any physical folding free energy. This notebook reproduces the pipeline on one structure and tests whether those numbers are **steric clash artefacts** rather than energies.

Three protocols on the same relaxed pose:

	what it does
A	the pipeline: `mutate_residue(pack_radius=8)`, side chains only
fa_rep	how much of the ddG is the Lennard-Jones repulsive term
B	A plus backbone + side-chain minimisation of the 8 A shell

Sign convention follows the paper: **ddG = E(WT) - E(mut)**, so *negative = destabilising*.

In [ ]:
```python
import os, sys, time, json, subprocess, urllib.request

def ensure_pyrosetta():
    """PyRosetta is not on PyPI; install via the RosettaCommons installer (free for academic use)."""
    try:
        import pyrosetta  # noqa: F401
        return "already installed"
    except ImportError:
        subprocess.run([sys.executable, "-m", "pip", "install", "-q",
```

```
"pyrosetta-installer"], check=True)
        import pyrosetta_installer
        pyrosetta_installer.install_pyrosetta()
        return "installed now (~1.5 GB)"

pyrosetta_status = ensure_pyrosetta()
print("PyRosetta:", pyrosetta_status)
```

```
PyRosetta: already installed
```

In [ ]:
```
WORK = "/tmp/ddg"
RAW = "https://raw.githubusercontent.com/chervov/ProteinModelling1/main/"
STRUCTURE_URL = RAW + "TYK2/af3/tyk2_af3_closed.pdb"
# The relaxed pose is published, so a fresh box skips the ~35 min
FastRelax.
RELAXED_URL = RAW + "data/ddg/TYK2_relaxed_closed.pdb.gz"
SHELL = 8.0          # repack radius, as in pipeline/ddg/worker.py

# Three positions with the most extreme stored ddG (all glycine), two
ordinary ones.
TEST_POSITIONS = [746, 964, 801, 667, 775]
TEST_ALTS = ["W", "F", "A", "G"]

os.makedirs(WORK, exist_ok=True)
print(f"{len(TEST_POSITIONS)} positions x {len(TEST_ALTS)} substitutions")
```

```
5 positions x 4 substitutions
```

In [ ]:
```
import pyrosetta
from pyrosetta import pose_from_pdb, get_fa_scorefxn
from pyrosetta.toolbox import mutate_residue
from pyrosetta.rosetta.core.scoring import ScoreType
from pyrosetta.rosetta.core.kinematics import MoveMap
from pyrosetta.rosetta.protocols.minimization_packing import MinMover
from pyrosetta.rosetta.protocols.relax import FastRelax
from pyrosetta.rosetta.protocols.constraint_generator import (
    CoordinateConstraintGenerator, AddConstraints)

pyrosetta.init("-mute all -ex1 -ex2aro -ignore_unrecognized_res "
               "-ignore_zero_occupancy false -detect_disulf false")
scorefxn = get_fa_scorefxn()
print("scorefunction:", scorefxn.get_name())
```

```
┌──────────────────────────────────────────────────────────────────────────────
────┐
│                                   PyRosetta-4
│
│                Created in JHU by Sergey Lyskov and PyRosetta Team
│
│                (C) Copyright Rosetta Commons Member Institutions
│
│
│
│ NOTE: USE OF PyRosetta FOR COMMERCIAL PURPOSES REQUIRES PURCHASE OF A LICE
NSE │
│          See LICENSE.PyRosetta.md or email license@uw.edu for details
│
└──────────────────────────────────────────────────────────────────────────────
────┘
PyRosetta-4 2026 [Rosetta PyRosetta4.Release.python313.ubuntu 2026.39+releas
e.49e05e797ca4383c6038f090e5db6c2593cb4963 2026-09-21T09:15:10] retrieved fr
om: http://www.pyrosetta.org
scorefunction: ref2015
```

In [ ]:
```
mo.md("""
## 1. Structure

The same AlphaFold-3 closed-state model the production run used, relaxed
with **FastRelax
under coordinate constraints** so it stays near the input geometry. The
relaxed pose is
cached on disk: the first run costs ~35 min for 1187 residues, afterwards
it is instant.

Watch the **total REU**: that single number is the yardstick for judging
every ddG below.
""")
```

1. Structure

The same AlphaFold-3 closed-state model the production run used, relaxed with **FastRelax under coordinate constraints** so it stays near the input geometry. The relaxed pose is cached on disk: the first run costs ~35 min for 1187 residues, afterwards it is instant. Watch the **total REU**: that single number is the yardstick for judging every ddG below.

In [ ]:
```
def load_relaxed_pose():
    """Return the relaxed pose, cheapest route first.

    1. local cache, 2. the relaxed pose published in the repo, 3. relax it
here (~35 min).
    Relaxation is FastRelax under coordinate constraints, exactly as
pipeline/ddg/worker.py.
    """
    cached = f"{WORK}/TYK2_relaxed_closed.pdb"
```

```
    if os.path.exists(cached) and os.path.getsize(cached) > 100_000:
        return pose_from_pdb(cached), "local cache"
    try:
        import gzip
        with urllib.request.urlopen(RELAXED_URL, timeout=300) as r:
            open(cached, "wb").write(gzip.decompress(r.read()))
        return pose_from_pdb(cached), "downloaded the published relaxed
pose"
    except Exception as exc:
        print("download failed, relaxing locally:", exc)
    src = f"{WORK}/tyk2_af3_closed.pdb"
    if not os.path.exists(src):
        urllib.request.urlretrieve(STRUCTURE_URL, src)
    p = pose_from_pdb(src)
    gen = CoordinateConstraintGenerator()
    add = AddConstraints(); add.add_generator(gen); add.apply(p)
    sf_cst = scorefxn.clone();
sf_cst.set_weight(ScoreType.coordinate_constraint, 1.0)
    t0 = time.time(); FastRelax(sf_cst, 1).apply(p)
    p.dump_pdb(cached)
    return p, f"relaxed here in {time.time() - t0:.0f}s"

pose, pose_note = load_relaxed_pose()
total_wt = scorefxn(pose)
print(f"{pose.total_residue()} residues, {total_wt:.0f} REU total "
      f"({total_wt / pose.total_residue():.2f} REU/residue) - {pose_note}")
```

```
1187 residues, -2689 REU total (-2.27 REU/residue) - local cache
```

In [ ]:
```
mo.md("""
## 2. Computing ddG three ways

For each substitution we compute the pipeline's value, the same thing with
a mobile
backbone, and how much of it is pure steric overlap.

The three extreme positions (**746, 964, 801**) are all **glycine** -
chosen because they
carry the largest ddG in the stored results. **667** and **775** are
ordinary controls.
""")
```

2. Computing ddG three ways

For each substitution we compute the pipeline's value, the same thing with a mobile backbone, and how much of it is pure steric overlap. The three extreme positions (**746, 964, 801**) are all **glycine** - chosen because they carry the largest ddG in the stored results. **667** and **775** are ordinary controls.

In [ ]:
```
def weighted_terms(p):
    """Weighted score terms of a pose, as a dict (fa_rep = Lennard-Jones
repulsion)."""
    scorefxn(p)
```

```
    totals = p.energies().total_energies()
    return {st.name: totals[st] * scorefxn.get_weight(st)
            for st in scorefxn.get_nonzero_weighted_scoretypes()}

def minimise_shell(p, position, radius=SHELL):
    """Protocol B: let the backbone AND side chains near the mutation move.
    This is the degree of freedom a repack-only protocol lacks, and the one
Flex ddG
    says bulky substitutions need."""
    mm = MoveMap()
    centre = p.residue(position).xyz("CA")
    for i in range(1, p.total_residue() + 1):
        if p.residue(i).has("CA") and (p.residue(i).xyz("CA") -
centre).norm() <= radius:
            mm.set_chi(i, True); mm.set_bb(i, True)
    MinMover(mm, scorefxn, "lbfgs_armijo_nonmonotone", 0.01, True).apply(p)
    return p

def ddg_for(position, alt):
    """ddG of one substitution under both protocols.

    Returns, for `<native><position><alt>`:
      A_repack_only  ddG from the pipeline protocol (side chains only).
      B_flexible_bb  ddG after backbone + side-chain minimisation of the
shell.
      fa_rep_share   fraction of the destabilisation that is pure steric
repulsion.
                     1.0 means the whole number is atoms overlapping, not
chemistry.

    Sign: ddG = E(WT) - E(mutant), so NEGATIVE = destabilising (the paper's
convention).
    The WT is repacked identically, so neither side is favoured.
    """
    native = pose.residue(position).name1()
    if native == alt:
        return None
    wt = pose.clone(); mutate_residue(wt, position, native,
pack_radius=SHELL)
    mut = pose.clone(); mutate_residue(mut, position, alt,
pack_radius=SHELL)
    a = scorefxn(wt) - scorefxn(mut)
    d_rep = weighted_terms(mut).get("fa_rep", 0.0) -
weighted_terms(wt).get("fa_rep", 0.0)
    b = scorefxn(minimise_shell(wt.clone(), position)) -
scorefxn(minimise_shell(mut.clone(), position))
    # a is negative when destabilising and d_rep positive, so negate to get
a 0-1 share
    share = round(-d_rep / a, 2) if abs(a) > 1e-9 else None
    return {"variant": f"{native}{position}{alt}", "pos": position, "wt":
native, "alt": alt,
            "A_repack_only": round(a, 1), "B_flexible_bb": round(b, 1),
"fa_rep_share": share}
```

In [ ]:
```
print(f"{'variant':8s} {'A (repack only)':>16s} {'B (flexible bb)':>16s}
{'steric':>8s}")
```

```
print(f"{'':8s} {'pipeline value':>16s} {'after bb moves':>16s}
{'share':>8s}")
print("-" * 52)

results = []
for _pos in TEST_POSITIONS:
    for _alt in TEST_ALTS:
        _rec = ddg_for(_pos, _alt)
        if _rec:
            results.append(_rec)
            _s = f"{_rec['fa_rep_share']:.0%}" if _rec["fa_rep_share"] is
not None else "-"
            print(f"{_rec['variant']:8s} {_rec['A_repack_only']:16.1f} "
                  f"{_rec['B_flexible_bb']:16.1f} {_s:>8s}", flush=True)

print("-" * 52)
print(f"{len(results)} variants. More negative = more destabilising.")
print("A collapsing to B means the ddG was atoms overlapping, not a real
energy cost.")
```

```
variant   A (repack only)  B (flexible bb)   steric
           pipeline value   after bb moves    share
----------------------------------------------------
G746W             -3006.4           -101.9     100%
G746F             -2924.0           -309.1     100%
G746A              -231.6            -13.6      96%
G964W             -2739.7            -46.2     100%
G964F             -2484.1           -109.3     100%
G964A              -149.4             -7.7      98%
G801W             -3060.4           -171.0     101%
G801F             -2211.2           -113.0     101%
G801A               -59.3             -0.4     111%
H667W             -2246.9            -41.4     100%
H667F              -12.6             -7.8     104%
H667A               -4.5             -3.8     -28%
H667G               -9.0             -8.4     -19%
R775W                1.8              3.1    -229%
R775F               -0.1             -0.8   -1241%
R775A                1.5              0.8      -4%
R775G               -0.7             -2.4    -318%
----------------------------------------------------
17 variants. More negative = more destabilising.
A collapsing to B means the ddG was atoms overlapping, not a real energy cos
t.
```

In [ ]:

```
mo.md("""
## 3. Results

`shrink_factor` is A divided by B: how many times smaller the ddG becomes
once the backbone
is allowed to relax. A factor near 1 means the protocol made no difference;
a large factor
means the original number was a clash the protocol could not resolve.
""")
```

3. Results

`shrink_factor` is A divided by B: how many times smaller the ddG becomes once the backbone is allowed to relax. A factor near 1 means the protocol made no difference; a large factor means the original number was a clash the protocol could not resolve.

In [ ]:
```
import pandas as pd

ddg_table = pd.DataFrame(results).sort_values("A_repack_only")
ddg_table["shrink_factor"] = (ddg_table.A_repack_only /
ddg_table.B_flexible_bb).round(1)
mo.ui.table(ddg_table, selection=None)
```

Search...

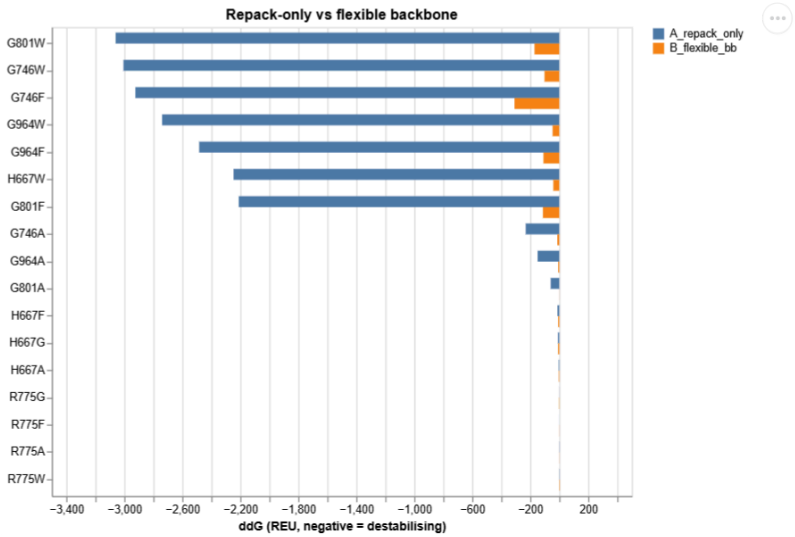
Index0 int64	variant str unique: 17	pos int64 667 775 964	wt str unique: 3	alt str unique: 4	A_repack_only float64 −3060.4 1.8	B_flexible_bb float64 −309.1 3.1	fa_rep_share float64 −12.41 1.11	shrink_factor float64 0.1 148.2
6	G801W	801	G	W	-3,060.4	-171.0	1.01	17.9
0	G746W	746	G	W	-3,006.4	-101.9	1.00	29.5
1	G746F	746	G	F	-2,924.0	-309.1	1.00	9.5
3	G964W	964	G	W	-2,739.7	-46.2	1.00	59.3
4	G964F	964	G	F	-2,484.1	-109.3	1.00	22.7
9	H667W	667	H	W	-2,246.9	-41.4	1.00	54.3
7	G801F	801	G	F	-2,211.2	-113.0	1.01	19.6
2	G746A	746	G	A	-231.6	-13.6	0.96	17.0
5	G964A	964	G	A	-149.4	-7.7	0.98	19.4
8	G801A	801	G	A	-59.3	-0.4	1.11	148.2

17 rows, 8 columns

No selection

In [ ]:
```
import altair as alt

_long = ddg_table.melt(id_vars="variant", value_vars=["A_repack_only",
"B_flexible_bb"],
                       var_name="protocol", value_name="ddG")
chart = alt.Chart(_long).mark_bar().encode(
    y=alt.Y("variant:N", sort=list(ddg_table.variant), title=None),
    x=alt.X("ddG:Q", title="ddG (REU, negative = destabilising)"),
    color=alt.Color("protocol:N", title=None),
    yOffset="protocol:N",
).properties(height=420, width=520, title="Repack-only vs flexible
backbone")
mo.ui.altair_chart(chart)
```



In [ ]:
```
mo.md("""
## 4. Cross-check: Rosetta's own ddG application

The obvious suspicion is that the repack-only protocol is at fault. To test
it we run
**`ddGMover`**, which *is* `ddg_monomer` (Kellogg et al. 2011): it ramps
the repulsive term,
minimises under coordinate constraints and takes the best of several decoys
- everything the
pipeline protocol lacks.

Results below are the stored run (5 variants, ~6 min each). Call
`run_ddg_monomer(position, alt)` to recompute any of them yourself.

Note the sign: **Rosetta positive = destabilising**, the opposite of the
paper's convention.
""")
```

4. Cross-check: Rosetta's own ddG application

The obvious suspicion is that the repack-only protocol is at fault. To test it we run **`ddGMover`**, which *is* `ddg_monomer` (Kellogg et al. 2011): it ramps the repulsive term, minimises under coordinate constraints and takes the best of several decoys - everything the pipeline protocol lacks. Results below are the stored run (5 variants, ~6 min each). Call `run_ddg_monomer(position, alt)` to recompute any of them yourself. Note the sign: **Rosetta positive = destabilising**, the opposite of the paper's convention.

In [ ]:
```
def run_ddg_monomer(position, alt, iterations=3):
    """Rosetta's own ddG protocol for one substitution. Returns (REU,
kcal/mol, seconds).

    ~6 min per variant, so the table below uses the stored run instead of
recomputing.
    """
```

```
    from pyrosetta.rosetta.protocols.ddg import ddGMover
    from pyrosetta.rosetta.utility import vector1_core_chemical_AA
    from pyrosetta.rosetta.core.chemical import aa_from_oneletter_code
    muts = vector1_core_chemical_AA(pose.total_residue())
    for i in range(1, pose.total_residue() + 1):
        muts[i] = pose.residue(i).aa()
    muts[position] = aa_from_oneletter_code(alt)
    mover = ddGMover(scorefxn, scorefxn, muts)
    mover.set_min_cst(True); mover.set_min(True); mover.set_mean(False)
    mover.restrict_to_nbrs(True); mover.num_iterations(iterations)
    work = pose.clone()
    t0 = time.time(); mover.apply(work)
    reu = mover.ddG()
    return reu, reu / 2.94, time.time() - t0   # 2.94 = documented REU ->
kcal/mol factor

kellogg_table = pd.DataFrame(
    json.load(urllib.request.urlopen(RAW +
"data/ddg/kellogg_ddg_monomer.json", timeout=120))
)
mo.ui.table(kellogg_table, selection=None)
```

Search...

variant str	pipeline_ddg float64	kellogg_REU float64	kellogg_kcal float64	seconds int64
G746W	-2,988.6	2,832.20	963.33	453
G746A	-228.8	216.58	73.67	338
G801W	-3,047.9	1,879.06	639.14	418
H667F	-17.8	6.11	2.08	342
R775A	1.5	1.83	0.62	339

5 rows, 5 columns

No selection

In [ ]:

```
_extreme = ddg_table[ddg_table.A_repack_only < -500]
mo.md(f"""
## What this shows

**The extreme values are not energies.** The whole folded protein scores **
{total_wt:.0f} REU**
over {pose.total_residue()} residues. A ddG of -3000 therefore puts the
mutant at
**+{-total_wt - 3000:.0f} REU** - a positive total energy, which no folded
structure has.

- **{len(_extreme)} of {len(ddg_table)}** variants fall below -500 under
the pipeline protocol, and
  their mean `fa_rep` share is **{_extreme.fa_rep_share.mean():.0%}**: the
ddG *is* the repulsive
  term and nothing else.
- Every extreme position is a **glycine**; G->A at the same sites is 10-50x
milder.
- The ordinary position (**775**) is unaffected by the protocol choice.

**But the protocol is not the culprit.** Rosetta's own `ddg_monomer`
(section 4) returns
**+2832 REU for G746W** - the same magnitude as the pipeline's -2989 -
while giving textbook
values for ordinary mutations (**H667F 2.08 kcal/mol**, **R775A 0.62
```

```
kcal/mol**). Two independently
written protocols agree, so there is no implementation defect. These
positions are buried glycines
that simply cannot host a tryptophan in a fixed backbone, and every
standard method reports that
as a saturated "does not fit" flag.

**What to do with the values.** Rank them; never quote them as energies.
That is what
`conclusions_ddG_difference_feature.md` (SS8-10) concludes and what the
pipeline README already
said: *"only the ordering is meaningful"*.
""")
```

What this shows

The extreme values are not energies. The whole folded protein scores **-2689 REU** over 1187 residues. A ddG of -3000 therefore puts the mutant at **+-311 REU** - a positive total energy, which no folded structure has.

- **7 of 17** variants fall below -500 under the pipeline protocol, and their mean `fa_rep` share is **100%**: the ddG *is* the repulsive term and nothing else.
- Every extreme position is a **glycine**; G->A at the same sites is 10-50x milder.
- The ordinary position (**775**) is unaffected by the protocol choice.

But the protocol is not the culprit. Rosetta's own `ddg_monomer` (section 4) returns **+2832 REU for G746W** - the same magnitude as the pipeline's -2989 - while giving textbook values for ordinary mutations (**H667F 2.08 kcal/mol**, **R775A 0.62 kcal/mol**). Two independently written protocols agree, so there is no implementation defect. These positions are buried glycines that simply cannot host a tryptophan in a fixed backbone, and every standard method reports that as a saturated "does not fit" flag. **What to do with the values.** Rank them; never quote them as energies. That is what `conclusions_ddG_difference_feature.md` (SS8-10) concludes and what the pipeline README already said: *"only the ordering is meaningful"*.